Performance Testing

Date	14 July, 2026
Team ID	
Project Name	Credit Card Approval Prediction (Flask App — app.py)
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter, Postman
Type of Testing	Load Testing, Stress Testing
Target Modules	Flask routes defined in app.py: / (application form), /predict (scoring API), /dashboard, /api/metrics, /sandbox, /notifications
Test Environment	Local System (Windows 11, Python 3.11, Flask dev server, app.run(host="0.0.0.0", port=7860, debug=True))
Model Artifacts Loaded at Startup	best_model.joblib, scaler.joblib, feature_columns.joblib, shap_background.joblib (loaded once — not per-request)
Test Date	14 July, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	Route	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits a credit card application form	/predict	1	30	Decision (Approve/Decline), probability and SHAP explanation generated within 2 seconds
2	Continuous prediction requests from analysts	/predict	100	120	Stable response time, no crashes, correct borderline-band flags (0.40–0.60) queued

3	Peak load testing on the scoring API	/predict	200	180	Application remains responsive and maintains acceptable performance under concurrency
S.No	**Test Scenario / Description**	**Route**	**No. of Virtual Users**	**Duration (sec)**	**Expected Outcome**
4	Compliance officer/analyst repeatedly loads the dashboard (model comparison + income-vs-approval charts)	/dashboard, /api/metrics	50	90	Charts render correctly; /api/metrics recomputes income bins from applicants.csv without errors
5	Customer explores what-if scenarios in the sandbox	/sandbox	50	90	Repeated scoring calls stay fast; nothing is persisted to disk or the notification queue
6	Analyst reviews the borderline-case notification queue while /predict is under load	/notifications	20	60	In-memory NOTIFICATIONS list renders consistently with no lost or duplicated entries

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass/F	ail)Remarks
1	Response Time (Avg) — /predict	< 2 seconds	1.35 sec	Pass	SHAP explanation adds modest overhead vs. a bare model call
2	Response Time (Max) — /predict	< 5 seconds	3.94 sec	Pass	Max seen on first request after startup (model already warm, no reload)
3	Throughput (Req/sec) — /predict	> 40 req/sec	48 req/sec	Pass	Slightly lower than a plain classifier due to explain() step
4	Response Time (Avg) — /dashboard, /api/metrics	< 2 seconds	1.62 sec	Pass	pandas.cut/groupby over applicants.csv on every call adds latency

5	Error Rate (all routes)	< 1%	0%	Pass	No request failures observed
6	CPU Utilization	< 80%	66%	Pass	Efficient CPU usage under peak load
7	Memory Utilization	< 80%	59%	Pass	Stable; no leak from the in-memory NOTIFICATIONS list during the test window

Step 4: Observations & Analysis

Key Findings

- The Credit Card Approval Prediction Flask app (app.py) successfully processed concurrent requests across/predict, /dashboard, /api/metrics, /sandbox and /notifications.
- Average /predict response time stayed below the 2-second target even with SHAP-based explanations enabled(falls back to global feature_importance from metrics.json if shap or shap_background.joblib is unavailable).
- No failed requests were observed; the borderline band (0.40 ≤ high_risk_probability ≤ 0.60) correctly appended entries to the in-memory NOTIFICATIONS list.
- The app remained stable under moderate and heavy workloads on the Flask development server.

Bottlenecks Identified

- Minor response-time increase beyond 200 concurrent users on /predict, mainly attributable to the shap.Explainercall inside explain().
- /api/metrics recomputes income-vs-approval bins from data/applicants.csv on every request instead of cachingthe result, adding avoidable latency under repeated dashboard refreshes.
- The NOTIFICATIONS list is a process-local Python list with no size cap or persistence, so it resets on restart andwould need a real datastore for multi-worker deployments.
- app.run(..., debug=True) is fine for local testing but is not representative of production performance; results shouldbe re-validated behind a production WSGI server (e.g. gunicorn).

Optimization Steps Taken / Recommended

- Model, scaler, feature_columns and the SHAP background/explainer are all loaded once at import time rather thanper-request.
- Reduced unnecessary preprocessing by building the one-hot feature row directly with build_feature_row() insteadof re-fitting encoders at request time.
- explain() gracefully falls back to precomputed feature_importance from metrics.json if the SHAP explainer fails,avoiding request failures.
- Recommended follow-up: cache /api/metrics results (e.g. short TTL) and move NOTIFICATIONS to a realdatabase or queue before scaling beyond a single worker.

Step 5: Screenshots / Evidence

Model comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Tree	0.73	0.9788	0.7313	0.8372	0.7807
Logistic Regression	0.7633	0.9852	0.7621	0.8594	0.8645
Random Forest	0.8592	0.9774	0.8718	0.9216	0.8611
XGBoost	0.9508	0.9569	0.993	0.9746	0.8403

Approval rate by income band